Brute Force

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Brute Force

A straightforward approach, usually based directly on the problem's statement and definitions of the concepts involved

Examples:

- 1. Computing a^n ($a>0$, n a nonnegative integer)
- 2. Computing n!
- 3. Multiplying two matrices
- 4. Searching for a key of a given value in a list

Brute-Force Sorting Algorithm

Selection Sort

Scan the array to find its smallest element and swap it with the first element. Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second elements. Generally, on pass i (0 ≤ i ≤ n-2), find the smallest element in A[i..n-1] and swap it with A[i]:

$$\underbrace{A_0 \le A_1 \le \cdots \le A_{i-1}}_{\text{in their final positions}} \;\Big|\; \underbrace{A_i, \ldots, A_{min}, \ldots, A_{n-1}}_{\text{the last } n-i \text{ elements}}$$

Example: 7 3 2 5

Analysis of Selection Sort

```
ALGORITHM  SelectionSort(A[0..n − 1])
    //Sorts a given array by selection sort
    //Input: An array A[0..n − 1] of orderable elements
    //Output: Array A[0..n − 1] sorted in ascending order
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min]   min ← j
        swap A[i] and A[min]
```

Time efficiency: O(n^2)

Space efficiency: O(n)

Stability: No

Brute-Force String Matching

pattern: a string of m characters to search for
text: a (longer) string of n characters to search in
problem: find a substring in the text that matches the pattern

Brute-force algorithm
Step 1: Align pattern at beginning of text
Step 2: Moving from left to right, compare each character of pattern to the corresponding character in text until

- all characters are found to match (successful search); or
- a mismatch is detected

Step 3: While pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat **Step 2**

Examples of Brute-Force String Matching

Pattern:
001011

Text:
1001010110100110010111101 0

Pattern: happy

Text: It is never too late to have a happy childhood.

Pseudocode and Efficiency

```
ALGORITHM  BruteForceStringMatch(T[0..n − 1], P[0..m − 1])
    //Implements brute-force string matching
    //Input: An array T[0..n − 1] of n characters representing a text and
    //        an array P[0..m − 1] of m characters representing a pattern
    //Output: The index of the first character in the text that starts a
    //        matching substring or −1 if the search is unsuccessful
    for i ← 0 to n − m do
        j ← 0
        while j < m and P[j] = T[i + j] do
            j ← j + 1
        if j = m return i
    return −1
```

Efficiency:

Brute-Force Polynomial Evaluation

Problem: Find the value of polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

at a point $x = x_0$

Brute-force algorithm

```
p ← 0.0
for i ← n downto 0 do
    power ← 1
        for j ← 1 to i do  //compute x^i
            power ← power * x
        p ← p + a[i] * power
return p
```

Efficiency:

Polynomial Evaluation: Improvement

We can do better by evaluating from right to left:

Better brute-force algorithm

```
p ← a[0]
power ← 1
for i ← 1 to n do
    power ← power * x
    p ← p + a[i] * power
return p
```

Efficiency:

Brute-Force Strengths and Weaknesses

- **Strengths**
  - wide applicability
  - simplicity
  - yields reasonable algorithms for some important problems (e.g., matrix multiplication, sorting, searching, string matching)

- **Weaknesses**
  - rarely yields efficient algorithms
  - some brute-force algorithms are unacceptably slow
  - not as constructive as some other design techniques

Exhaustive Search

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Exhaustive Search

A brute force solution to a problem involving the search for an element with a special property, usually among combinatorial objects such as permutations, combinations, or subsets of a set.

Method:

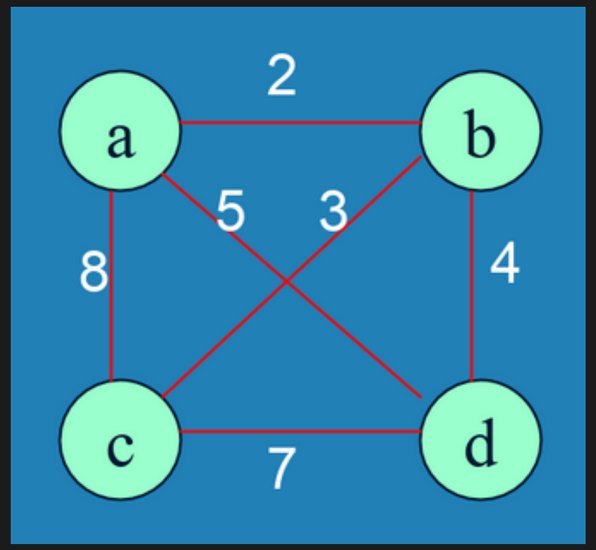
- Generate a list of all potential solutions to the problem in a systematic manner
- Evaluate potential solutions one by one, disqualifying infeasible ones and, for an optimization problem, keeping track of the best one found so far
- When search ends, announce the solution(s) found

Example 1: Traveling Salesman Problem

Given n cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city

Alternatively: Find shortest Hamiltonian circuit in a weighted connected graph

Example:



TSP by Exhaustive Search

Tour	Cost
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

More tours?

Less tours?

Efficiency: Ω(2^n)

Example 2: Knapsack Problem

Given n items:

- weights: w_1 w_2 ... w_n
- values: v_1 v_2 ... v_n
- a knapsack of capacity W

Find most valuable subset of the items that fit into the knapsack

Example: Knapsack capacity W=16

item	weight	value
1	2	$20
2	5	$30
3	10	$50
4	5	$10

Knapsack Problem by Exhaustive Search

Subset	***Total weight***	***Total value***
{1}	2	$20
{2}	5	$30
{3}	10	$50
{4}	5	$10
{1,2}	7	$50
{1,3}	12	$70
{1,4}	7	$30
{2,3}	15	$80
{2,4}	10	$40
{3,4}	15	$60
{1,2,3}	17	not feasible
{1,2,4}	12	$60
{1,3,4}	17	not feasible
{2,3,4}	20	not feasible
{1,2,3,4}	22	not feasible

Efficiency: Ω(2^n)

Example 3: The Assignment Problem

There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is C[i,j]. Find an assignment that minimizes the total cost.

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

Algorithmic Plan: Generate all legitimate assignments, compute their costs, and select the cheapest one.

How many assignments are there?

Assignment Problem by Exhaustive Search

Pose the problem as the one about a cost matrix:

Assignment (col.#s)	Total Cost
1, 2, 3, 4	9+4+1+4=18
1, 2, 4, 3	9+4+8+9=30
1, 3, 2, 4	9+3+8+4=24
1, 3, 4, 2	9+3+8+6=26
1, 4, 2, 3	9+7+8+9=33
1, 4, 3, 2	9+7+1+6=23
etc.	

$$C = \begin{matrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{matrix}$$

Efficiency: Ω(2^n)

Final Comments on Exhaustive Search

- Exhaustive-search algorithms run in a realistic amount of time only on very small instances.

- In some cases, there are much better alternatives!
  - Euler circuits
  - shortest paths
  - minimum spanning tree
  - assignment problem

- In many cases, exhaustive search or its variation is the only known way to get an exact solution.

Q & A

FOR QUESTIONS OR COMMENTS

EMAIL ADDRESS

ipineda@yachaytech.edu.ec